BÁO CÁO ĐỀ TÀI

Xây dựng Point-in-Time Correct Feature Store

Bộ dữ liệu minh họa: IEEE-CIS Fraud Detection
Thời gian thực hiện: 5 tuần
Tiến độ hiện tại: Tuần 2 — đã xây dựng và kiểm thử nguyên mẫu trên máy local
Lĩnh vực: Data Engineering và Machine Learning
Ngày cập nhật: 13/11/2026

---

1. Tóm tắt đề tài

Đề tài xây dựng một hệ thống tạo đặc trưng lịch sử cho mô hình học máy. Yêu cầu quan trọng nhất là: khi dự đoán một giao dịch ở thời điểm nào, hệ thống chỉ được sử dụng thông tin đã có trước thời điểm đó.

Ví dụ, cần dự đoán giao dịch xảy ra lúc 10:00:

```
08:00  giao dịch cũ       → được sử dụng
09:30  giao dịch cũ       → được sử dụng
10:00  giao dịch hiện tại → không tính vào lịch sử của chính nó
11:00  giao dịch tương lai → không được sử dụng
```

Nếu dữ liệu lúc 11:00 được dùng để dự đoán giao dịch lúc 10:00, kết quả đánh giá sẽ không còn đáng tin cậy. Khi chạy ngoài thực tế lúc 10:00, giao dịch 11:00 chưa xảy ra.

Hệ thống được kiểm chứng bằng bài toán phát hiện gian lận giao dịch. Tuy nhiên, mục tiêu chính của đề tài là xây dựng quy trình dữ liệu đúng theo thời gian, không phải tìm mô hình gian lận mạnh nhất.

---

2. Chín mục tiêu theo chín mốc thực hiện

Đề tài gồm 9 mốc và có 9 mục tiêu tương ứng:

Mốc	Mục tiêu
1	Hiểu và kiểm tra chất lượng dữ liệu đầu vào
2	Chọn khóa đại diện để nhóm các giao dịch có liên quan
3	Xây nền tảng dữ liệu Bronze và Silver
4	Định nghĩa các đặc trưng và tạo bộ dữ liệu mẫu để kiểm tra bằng tay
5	Tính đặc trưng lịch sử đúng theo thời điểm dự báo
6	Lưu đặc trưng thành phiên bản và hỗ trợ tính lại dữ liệu cũ
7	Xây dữ liệu huấn luyện, train mô hình và đánh giá kết quả
8	Kiểm tra rò rỉ dữ liệu, trường hợp đặc biệt và độ an toàn của pipeline
9	Hoàn thiện tài liệu, báo cáo, demo và chạy thử từ môi trường sạch

Phần tiếp theo trình bày công việc và kết quả của từng mốc theo đúng thứ tự này.

---

3. Dữ liệu sử dụng

Đề tài sử dụng bộ dữ liệu IEEE-CIS Fraud Detection.

Dữ liệu gồm bốn file chính:

```
train_transaction.csv
train_identity.csv
test_transaction.csv
test_identity.csv
```

3.1. Bảng giao dịch

Bảng giao dịch chứa:

- mã giao dịch `TransactionID`;
- thời gian tương đối `TransactionDT`;
- số tiền `TransactionAmt`;
- nhóm sản phẩm `ProductCD`;
- thông tin thẻ, địa chỉ, email;
- nhãn `isFraud` cho biết giao dịch có bị xác định là gian lận hay không.

`TransactionDT` là số giây tính từ một mốc thời gian ẩn, không phải ngày giờ thật. Project tạo một mốc giả cố định để đổi nó thành `event_time`. Việc này chỉ phục vụ sắp xếp, chia thời gian và tính cửa sổ lịch sử.

3.2. Bảng thông tin nhận dạng

Bảng identity chứa một số thông tin về thiết bị, trình duyệt, hệ điều hành và kết nối mạng. Không phải giao dịch nào cũng có identity, vì vậy hệ thống giữ lại toàn bộ giao dịch và đánh dấu giao dịch nào có thông tin nhận dạng.

---

4. Công việc đã thực hiện theo 9 mốc

Mốc 1 — Hiểu và kiểm tra dữ liệu

Các công việc đã hoàn thành:

- đọc thông tin của từng file;
- đếm số dòng, số cột và dung lượng;
- kiểm tra cấu trúc cột;
- kiểm tra tính duy nhất của `TransactionID`;
- kiểm tra tỷ lệ gian lận;
- kiểm tra mức độ bao phủ của bảng identity;
- tính tỷ lệ thiếu và số giá trị khác nhau của từng cột;
- viết tài liệu giải thích dữ liệu;
- tạo mã SHA-256 cho từng raw file.

SHA-256 có thể hiểu là “dấu vân tay” của file. Nếu nội dung file thay đổi, dấu vân tay cũng thay đổi. Nhờ đó có thể biết pipeline đang dùng đúng bộ dữ liệu nào.

Kết quả: Có báo cáo tự động bằng JSON/Markdown, data dictionary và raw checksum manifest.

Mốc 2 — Chọn khóa đại diện cho giao dịch

Dataset không cung cấp mã khách hàng hoặc tài khoản rõ ràng. Vì vậy project cần tạo một khóa đại diện để nhóm các giao dịch có khả năng liên quan.

Hai cách nhóm được chọn:

Mức	Cách nhóm	Ý nghĩa
Chính	`card1 + addr1`	Nhóm theo thẻ và khu vực thanh toán
Bổ sung	`card1`	Nhóm rộng hơn theo thẻ, dùng được khi địa chỉ bị thiếu

Ví dụ:

```
Thẻ A + Hà Nội  → một nhóm chính
Thẻ A + TP.HCM  → một nhóm chính khác
Cả hai          → vẫn thuộc nhóm bổ sung Thẻ A
```

Việc lựa chọn dựa trên tỷ lệ giao dịch tạo được khóa, số giao dịch lặp lại, số nhóm chỉ xuất hiện một lần, độ lệch giữa nhóm lớn và nhỏ, cùng khoảng thời gian hoạt động của nhóm.

Kết quả: Mô hình nhận cả hai bộ đặc trưng. Nếu địa chỉ thiếu, giao dịch vẫn được giữ và bộ đặc trưng theo thẻ vẫn có thể sử dụng.

Mốc 3 — Xây nền tảng Bronze và Silver

Để dễ hiểu, có thể coi:

- **Bronze** là bản sao dữ liệu nguồn kèm thông tin dữ liệu được nhập từ đâu và lúc nào;
- **Silver** là dữ liệu đã được chuẩn hóa để các bước sau sử dụng thống nhất.

Bronze bổ sung:

```
mã lần chạy
tên nguồn
tên file
SHA-256 của file
thời gian nhập dữ liệu
số thứ tự dòng nguồn
```

Silver thực hiện:

- đổi kiểu dữ liệu;
- xử lý dòng trùng có kiểm soát;
- tạo thời gian sự kiện;
- nối bảng giao dịch với identity nhưng không làm mất giao dịch;
- tạo hai khóa nhóm ở Mốc 2;
- lưu dữ liệu dưới dạng Parquet theo thời gian;
- kiểm tra số dòng, mã giao dịch và kết quả nối bảng.

Nếu hai dòng có cùng `TransactionID` nhưng nội dung khác nhau, pipeline dừng để người dùng kiểm tra thay vì tự chọn một dòng.

Kết quả: Có một bảng giao dịch chuẩn dùng chung cho việc tạo đặc trưng và huấn luyện.

Mốc 4 — Định nghĩa đặc trưng và bộ kiểm tra mẫu

Project chọn 10 loại đặc trưng lịch sử:

Đặc trưng	Ý nghĩa dễ hiểu
Số giao dịch trong 1 giờ	Hoạt động rất gần thời điểm dự báo
Số giao dịch trong 24 giờ	Tần suất trong một ngày
Số giao dịch trong 7 ngày	Tần suất trong một tuần
Tổng tiền trong 1 giờ	Tổng giá trị giao dịch gần nhất
Tổng tiền trong 24 giờ	Tổng giá trị trong ngày
Tổng tiền trong 7 ngày	Tổng giá trị trong tuần
Tiền trung bình trong 24 giờ	Mức giao dịch trung bình gần đây
Giao dịch lớn nhất trong 7 ngày	Mức tiền cao nhất gần đây
Thời gian từ giao dịch trước	Giao dịch có diễn ra dồn dập không
Số nhóm sản phẩm trong 7 ngày	Mức độ đa dạng hoạt động

Mỗi đặc trưng được tính theo hai cách nhóm ở Mốc 2, tạo ra 20 cột.

Ví dụ:

```
card_addr_txn_count_24h → số giao dịch theo thẻ + địa chỉ
card_txn_count_24h      → số giao dịch theo thẻ
```

Một bộ dữ liệu nhỏ được tạo bằng tay, trong đó kết quả mong đợi cũng được tính bằng tay. Bộ này chứa các trường hợp như không có lịch sử, đúng biên 1 giờ/24 giờ/7 ngày, cùng thời điểm, thiếu khóa nhóm và có dữ liệu tương lai.

Kết quả: Có định nghĩa đặc trưng bằng YAML, bộ kiểm tra mẫu và kết quả chuẩn để đối chiếu.

Mốc 5 — Tính đặc trưng đúng theo thời gian

Mỗi giao dịch được xem là một điểm cần dự báo. Một giao dịch lịch sử chỉ được sử dụng nếu nó đứng trước giao dịch cần dự báo.

Khi hai giao dịch có cùng thời gian, `TransactionID` được dùng để xác định thứ tự:

```
ID 10 → chưa thấy giao dịch nào cùng thời điểm
ID 11 → thấy ID 10
ID 12 → thấy ID 10 và ID 11
```

Pipeline đã được kiểm tra rằng:

- giao dịch hiện tại không tính vào lịch sử của chính nó;

- giao dịch tương lai không ảnh hưởng giao dịch quá khứ;
- thêm dữ liệu tương lai không làm thay đổi đặc trưng cũ;
- thay đổi thứ tự dòng trong file không làm đổi kết quả;
- giao dịch thiếu khóa nhóm vẫn được giữ lại.

Kết quả: Tạo đủ 20 đặc trưng lịch sử và khớp với kết quả được tính bằng tay.

Mốc 6 — Lưu đặc trưng và hỗ trợ tính lại

Đặc trưng được lưu thành bảng Parquet theo từng ngày:

```
data/offline_store/transaction_features/v1/
  observation_date=YYYY-MM-DD/
```

`v1` là phiên bản đầu tiên. Khi đã công bố, phiên bản này không được ghi đè tùy ý.

Pipeline hỗ trợ **backfill**, tức tính lại một khoảng dữ liệu cũ. Ví dụ, một giao dịch xảy ra lúc 09:00 nhưng đến 12:00 hệ thống mới nhận được. Nếu muốn sửa đặc trưng của các giao dịch sau 09:00, người dùng chủ động chạy lại ngày bị ảnh hưởng.

Khi thay dữ liệu của một ngày:

```
tính vào thư mục tạm
→ kiểm tra kết quả
→ sao lưu dữ liệu ngày cũ
→ đưa dữ liệu mới vào
→ khôi phục bản cũ nếu có lỗi
```

Kết quả: Có bảng đặc trưng theo phiên bản, lưu theo ngày, hỗ trợ chạy lại mà không tạo dòng trùng.

Mốc 7 — Xây dữ liệu huấn luyện và đánh giá mô hình

Dữ liệu được chia theo thời gian thay vì chia ngẫu nhiên:

Tập	Số dòng	Tỷ lệ gian lận
Train	414.542	3,5220%
Validation	86.672	3,4025%
Internal test	89.326	3,4861%

Các giá trị dùng để xử lý dữ liệu thiếu và danh sách category chỉ được học từ train. Validation và test không tham gia vào bước này.

Hai mô hình CatBoost được huấn luyện:

Mô hình	Đầu vào
Raw baseline	15 thông tin gốc của giao dịch
Raw + PIT	15 thông tin gốc + 20 đặc trưng lịch sử + 20 cờ báo thiếu

CatBoost được chọn vì dữ liệu có nhiều cột dạng nhóm như thẻ, email, thiết bị và địa chỉ.

Vì sao chọn CatBoost?

CatBoost phù hợp với bản MVP vì ba lý do chính:

1. Dataset có 14 cột dạng phân loại như loại sản phẩm, thông tin thẻ, email và thiết bị. CatBoost có thể xử lý trực tiếp các cột này, không cần tạo hàng nghìn cột one-hot.
2. Gian lận chỉ chiếm khoảng 3,5% dữ liệu. CatBoost hỗ trợ tự cân bằng trọng số giữa giao dịch bình thường và giao dịch gian lận.
3. Cùng một thuật toán có thể dùng cho cả baseline và mô hình có PIT features. Nhờ đó, chênh lệch kết quả chủ yếu phản ánh giá trị của feature mới thay vì do đổi loại mô hình.

Random Forest cũng có thể dùng làm mô hình tham chiếu, nhưng cần thêm cách mã hóa category, có thể tạo dữ liệu rất rộng và không phải lựa chọn thuận tiện nhất trong thời hạn 5 tuần. Mục tiêu của đề tài không phải tìm thuật toán mạnh nhất mà là so sánh công bằng trước và sau khi thêm PIT features.

Cấu hình đã sử dụng

Tham số	Giá trị	Ý nghĩa và lý do chọn
`iterations`	300	Cho phép tối đa 300 vòng học; early stopping có thể dừng sớm
`learning_rate`	0,08	Mỗi vòng điều chỉnh vừa phải, cân bằng giữa tốc độ và độ ổn định
`depth`	7	Đủ khả năng học quan hệ phi tuyến nhưng chưa làm cây quá phức tạp
`l2_leaf_reg`	5,0	Phạt mô hình quá phức tạp để giảm overfitting
`loss_function`	`Logloss`	Hàm mất mát phù hợp bài toán phân loại nhị phân
`eval_metric`	`AUC`	Theo dõi khả năng xếp fraud cao hơn giao dịch bình thường
`auto_class_weights`	`Balanced`	Tăng ảnh hưởng của lớp fraud hiếm khi học
`early_stopping_rounds`	40	Dừng nếu AUC validation không cải thiện trong 40 vòng
`random_seed`	42	Giúp kết quả có thể tái tạo và hai mô hình được so sánh công bằng
`thread_count`	-1	Sử dụng các CPU thread khả dụng để giảm thời gian train

Hai mô hình dùng chính xác cùng cấu hình, cùng train/validation split và cùng seed. Baseline chỉ nhận 15 raw features; mô hình MVP nhận thêm 20 PIT features và 20 cờ báo thiếu.

Các tham số được tinh chỉnh như thế nào?

Trong phiên bản hiện tại chưa chạy grid search hoặc tối ưu hàng trăm tổ hợp tham số. Nhóm chọn một cấu hình CatBoost ở mức vừa phải cho MVP, sau đó dùng tập validation và early stopping để chọn số vòng học thực tế:

```
Raw baseline:    dừng ở iteration 140, validation AUC = 0,849364
Raw + PIT model: dừng ở iteration 296, validation AUC = 0,860614
```

Tập internal test hoàn toàn không tham gia chọn tham số hoặc chọn iteration; nó chỉ được mở ở bước đánh giá cuối. Cách làm này tránh điều chỉnh mô hình theo chính tập dùng để báo cáo kết quả. Vì chưa thực hiện hyperparameter search đầy đủ, báo cáo chỉ kết luận PIT features cải thiện kết quả trong cấu hình thử nghiệm hiện tại, không khẳng định đây là cấu hình CatBoost tối ưu.

Nếu còn thời gian ở tuần sau, có thể thử một không gian nhỏ trên validation, ví dụ `depth` từ 6 đến 8, `learning_rate` từ 0,05 đến 0,1 và `l2_leaf_reg` từ 3 đến 10. Vì kết quả internal test v1 đã được xem trong báo cáo này, phiên bản tuning tiếp theo phải khóa cấu hình trước khi đánh giá và cần một khoảng holdout mới nếu muốn đưa ra kết luận hoàn toàn độc lập.

Kết quả trên tập internal test chưa được dùng khi huấn luyện:

Chỉ số	Raw baseline	Raw + PIT
ROC-AUC	0,815880	**0,829082**
PR-AUC	0,236695	**0,254277**
Precision trong top 100	74%	**83%**
Precision trong top 1.000	43,7%	**45,3%**

Hiểu đơn giản, nếu chỉ kiểm tra 100 giao dịch có điểm rủi ro cao nhất, mô hình có đặc trưng lịch sử tìm được 83 giao dịch gian lận, còn mô hình chỉ dùng dữ liệu gốc tìm được 74.

Tuy nhiên, mô hình có đặc trưng lịch sử không tốt hơn ở mọi ngưỡng. Báo cáo giữ nguyên những trường hợp raw baseline cao hơn nhẹ để tránh kết luận quá mức.

Kết quả: Đặc trưng lịch sử cung cấp thêm thông tin hữu ích và được đánh giá trên dữ liệu đến sau giai đoạn huấn luyện.

Mốc 8 — Kiểm tra độ đúng và độ an toàn

Project chạy 17 kiểm tra để xác nhận:

- không sử dụng giao dịch tương lai;
- không dùng nhãn gian lận để tạo đặc trưng;
- xử lý dữ liệu chỉ học từ train;
- ba tập thời gian không chồng lấn;
- không mất hoặc nhân đôi giao dịch.

Các trường hợp đặc biệt cũng được kiểm tra:

- thiếu khóa nhóm;
- một nhóm có rất nhiều giao dịch;
- nhiều giao dịch cùng thời điểm;
- dữ liệu quá khứ đến muộn;
- thêm dữ liệu tương lai.

Pipeline backfill được thu hẹp để chỉ xem xét các nhóm giao dịch liên quan. Trong bộ kiểm tra nhỏ, số dòng lịch sử cần xem xét giảm từ 10.000 xuống 1.440, tương đương 85,6%.

Thời gian chạy trên bộ nhỏ chưa nhanh hơn vì phải tạo thêm bảng trung gian:

```
Trước: 0,114 giây
Sau:   0,179 giây
```

Vì vậy, kết luận đúng là giảm lượng dữ liệu trung gian, chưa phải chứng minh chạy nhanh hơn ở mọi quy mô. Kết quả trước và sau tối ưu không có dòng nào khác nhau.

Nếu pipeline lỗi, dữ liệu tạm không được công bố và file log vẫn được giữ lại để kiểm tra.

Kết quả: Có bằng chứng về độ đúng, cách xử lý lỗi và sự tương đương trước/sau tối ưu.

Mốc 9 — Hoàn thiện tài liệu và chạy thử cuối

Đã hoàn thành:

- README hướng dẫn từ lúc clone project tới khi chạy hết pipeline;
- script PowerShell chạy tự động 15 bước;
- báo cáo tổng quan bằng Markdown và PDF;
- tài liệu giải thích từng phần đã làm.

Đang thực hiện:

- cập nhật sơ đồ kiến trúc và runbook cuối;
- làm slide;
- chuẩn bị demo dữ liệu tương lai bị loại;
- chạy thử toàn bộ project trên một thư mục clone sạch;
- dọn repository và tạo phiên bản phát hành cuối.

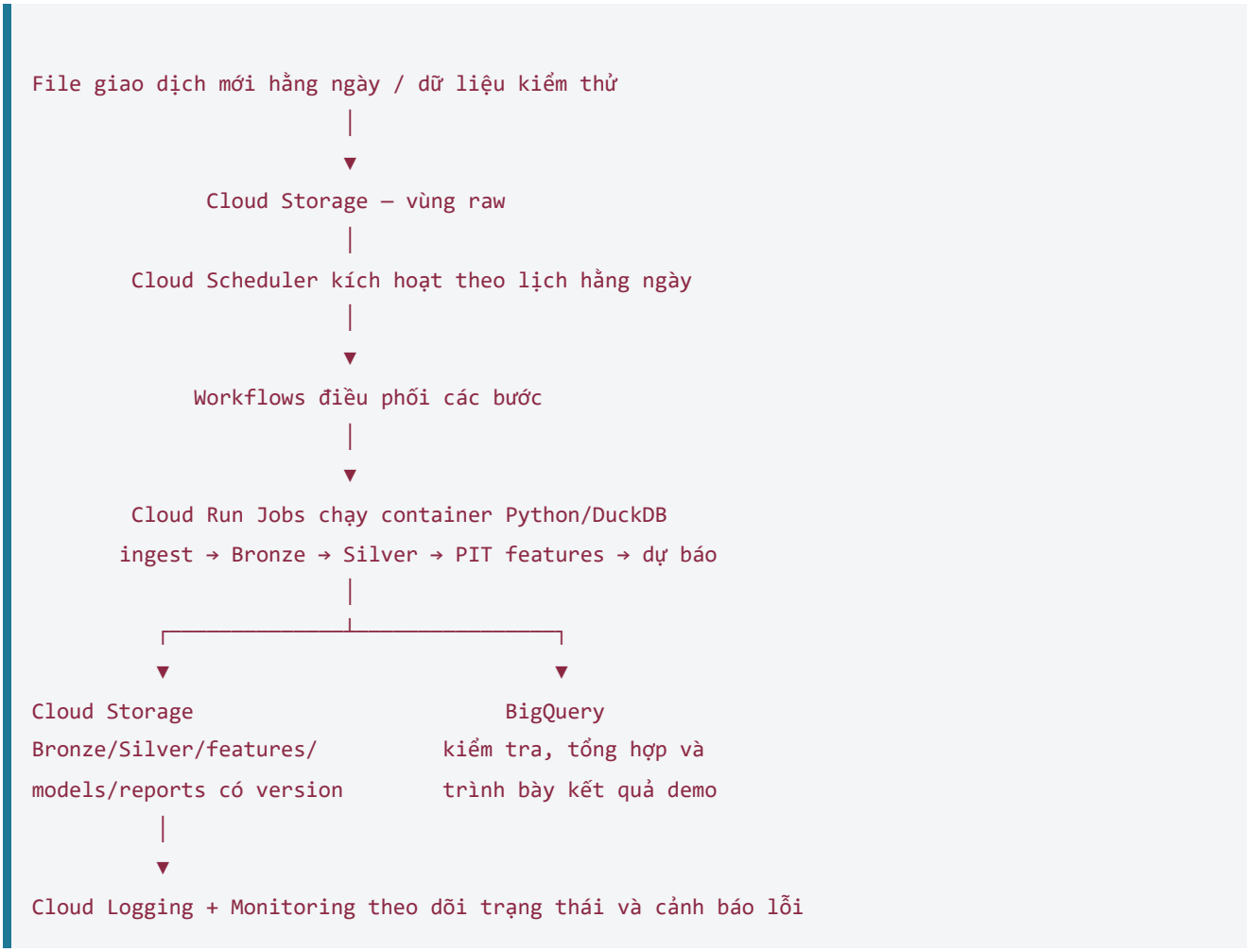
Kết quả hiện tại: Pipeline phát triển đã hoàn thành; bước xác nhận cuối trên môi trường sạch chưa được tuyên bố thành công cho tới khi lần chạy đó kết thúc.

---

5. Kế hoạch thực hiện trong các tuần tiếp theo

Chín mốc ở phần trên là chín nhóm công việc của sản phẩm, không có nghĩa là mỗi mốc tương ứng với một tuần. Toàn bộ đề tài kéo dài 5 tuần. Tại thời điểm viết báo cáo, project đang ở tuần 2: phần xử lý chính đã có nguyên mẫu chạy trên máy local, còn việc triển khai và vận hành trên Google Cloud là kế hoạch cho tuần 3 đến tuần 5.

5.1. Kiến trúc Google Cloud dự kiến



Mã nguồn vẫn được quản lý trên GitHub. Khi có phiên bản mới, Cloud Build sẽ chạy kiểm thử, đóng gói code thành Docker image và lưu image trong Artifact Registry. Cloud Run Jobs lấy đúng image đó để chạy, nhờ vậy môi trường trên cloud thống nhất với phiên bản code đã kiểm tra.

5.2. Công cụ dự kiến sử dụng và vai trò

Công cụ	Dùng để làm gì trong project	Lý do chọn
Cloud Storage	Lưu file raw, Bronze, Silver, offline features, model và report	Phù hợp dữ liệu dạng file CSV/Parquet và dễ chia theo ngày, phiên bản
Docker	Đóng gói code và thư viện thành một môi trường chạy thống nhất	Tránh tình trạng chạy được ở máy này nhưng lỗi ở máy khác
Artifact Registry	Lưu các phiên bản Docker image	Cloud Run có thể lấy đúng image đã được build và kiểm tra
Cloud Build	Tự động test, build và đẩy image khi cập nhật code	Giảm thao tác thủ công trước mỗi lần triển khai
Cloud Run Jobs	Chạy các batch job Python/DuckDB của pipeline	Tận dụng code hiện tại, không phải quản lý máy chủ liên tục
Workflows	Gọi các job theo đúng thứ tự và dừng khi một bước lỗi	Thể hiện rõ luồng chạy từ ingest tới feature và dự báo
Cloud Scheduler	Kích hoạt workflow hằng ngày	Mô phỏng tình huống mỗi ngày có một lô giao dịch mới
BigQuery	Truy vấn kết quả và tạo bảng tổng hợp phục vụ demo	Có thể đọc dữ liệu Parquet trên Cloud Storage hoặc nạp bảng cần truy vấn thường xuyên
Secret Manager và IAM	Quản lý thông tin nhạy cảm và quyền truy cập	Không đặt credential trực tiếp trong source code
Cloud Logging và Monitoring	Xem log, thời gian chạy, lỗi và tạo cảnh báo	Biết pipeline thành công hay thất bại mà không cần mở từng file log

Cloud Run Jobs phù hợp với project vì nó chạy workload dạng batch từ container, hỗ trợ retry, timeout và nhiều task khi cần. Workflows có thể được Cloud Scheduler gọi theo lịch. Cloud Run tự động chuyển log của container sang Cloud Logging. Các khả năng này được mô tả trong tài liệu chính thức của Cloud Run Jobs, Workflows và Cloud Scheduler và Cloud Run Logging.

Cloud Storage sẽ là nơi lưu dữ liệu chính. BigQuery có thể đọc trực tiếp file Parquet trên Cloud Storage thông qua external table; những bảng được truy vấn nhiều có thể nạp vào BigQuery để truy vấn nhanh và ổn định hơn. Cách này giúp project không cần chuyển toàn bộ dữ liệu sang một hệ quản trị mới ngay từ đầu. Tham khảo tài liệu chính thức về Cloud Storage, BigQuery external tables và nạp Parquet từ Cloud Storage.

Docker image được lưu trong Artifact Registry, còn API key hoặc thông tin nhạy cảm được lưu trong Secret Manager, không ghi vào Git.

5.3. Kế hoạch tuần 3

Mục tiêu: đưa pipeline local lên môi trường cloud và chạy được bằng tay.

Các công việc dự kiến:

1. Viết `Dockerfile` và kiểm tra pipeline chạy được trong container.
2. Tạo Google Cloud project, service account và quyền truy cập tối thiểu.
3. Tạo cấu trúc thư mục dữ liệu trên Cloud Storage:

```
raw/
bronze/
silver/
offline_features/v1/
training/v1/
models/v1/
reports/
```

1. Dùng Cloud Build để build image và lưu vào Artifact Registry.
2. Tạo Cloud Run Job cho pipeline, truyền vào ngày xử lý và phiên bản feature.
3. Chạy thử một ngày dữ liệu, kiểm tra row count, checksum, log và file đầu ra.

Đầu ra tuần 3: pipeline chạy thủ công trên Google Cloud và tạo kết quả tương đương local.

5.4. Kế hoạch tuần 4

Mục tiêu: biến các job riêng lẻ thành pipeline hằng ngày có thể theo dõi và chạy lại an toàn.

Các công việc dự kiến:

1. Tạo Workflows gọi lần lượt ingest, Bronze, Silver, PIT features và dự báo.
2. Tạo Cloud Scheduler kích hoạt workflow một lần mỗi ngày.
3. Truyền `processing_date` vào pipeline để mỗi lần chỉ xử lý ngày cần thiết.
4. Hoàn thiện backfill: chọn ngày cần tính lại, không ghi đè nhầm ngày khác.
5. Tạo BigQuery external table hoặc bảng tổng hợp để xem feature và kết quả dự báo.
6. Thêm log có cấu trúc gồm `run_id`, ngày xử lý, phiên bản code, số dòng và trạng thái.
7. Tạo cảnh báo khi job thất bại, thiếu file đầu vào hoặc kiểm tra chất lượng không đạt.

Ví dụ: sáng 06/11, Scheduler khởi động workflow xử lý dữ liệu ngày 05/11. Pipeline đọc lịch sử đến hết 04/11 để tạo đặc trưng cho từng giao dịch 05/11, ghi kết quả vào partition ngày 05/11 và không tính lại toàn bộ dữ liệu nếu không có yêu cầu backfill.

Đầu ra tuần 4: pipeline tự chạy theo lịch, có log, cảnh báo và có thể backfill một ngày cụ thể.

5.5. Kế hoạch tuần 5

Mục tiêu: kiểm thử toàn bộ hệ thống, hoàn thiện demo và bàn giao.

Các công việc dự kiến:

1. Chạy một lượt end-to-end trên môi trường cloud sạch.
2. So sánh số dòng, checksum và feature giữa local với cloud.
3. Demo tình huống thêm file giao dịch mới và xem pipeline tự tạo kết quả.

4. Demo thêm dữ liệu tương lai nhưng feature quá khứ không thay đổi.
5. Demo backfill một ngày bị thiếu và kiểm tra các ngày khác không bị ảnh hưởng.
6. Ghi nhận thời gian chạy, lượng dữ liệu đọc và chi phí thử nghiệm.
7. Hoàn thiện README, sơ đồ kiến trúc, báo cáo, slide và video hoặc kịch bản demo.

Đầu ra tuần 5: bản demo cloud, báo cáo kết quả, repository gọn và hướng dẫn tái chạy.

5.6. Phạm vi bắt buộc và phần mở rộng

Để phù hợp thời hạn, phần bắt buộc chỉ gồm Cloud Storage, Docker, Artifact Registry, Cloud Run Jobs, Workflows, Scheduler, Logging và một cách xem kết quả bằng BigQuery. Vertex AI Model Registry hoặc API dự báo thời gian thực bằng Cloud Run Service có thể làm nếu còn thời gian, nhưng không phải điều kiện để hoàn thành đề tài.

---

6. Kiến trúc xử lý dữ liệu hiện tại

```
CSV gốc
   ↓
Bronze: giữ dữ liệu nguồn và thông tin lần nhập
   ↓
Silver: dữ liệu đã chuẩn hóa và có thời gian sự kiện
   ↓
Tính đặc trưng lịch sử đúng theo thời điểm
   ↓
Offline feature store theo phiên bản và ngày
   ↓
Dữ liệu huấn luyện chia theo thời gian
   ↓
Mô hình và báo cáo đánh giá
   ↓
Kiểm tra rò rỉ dữ liệu và khả năng tái tạo
```

Các tên Bronze, Silver và feature store chỉ là tên gọi cho từng tầng xử lý. Ý nghĩa chính là:

```
Dữ liệu gốc
→ dữ liệu sạch
→ đặc trưng lịch sử
→ dữ liệu huấn luyện
→ kết quả đánh giá có thể kiểm tra lại
```

---

7. Những kết quả chính

1. Tạo được 20 đặc trưng lịch sử cho hai cách nhóm giao dịch.
2. Đặc trưng quá khứ không thay đổi khi thêm giao dịch tương lai.
3. Dữ liệu đến muộn được xử lý bằng backfill có chủ đích.
4. Training dataset có 590.540 giao dịch, không thiếu dòng khi nối với bảng đặc trưng.
5. Mô hình có đặc trưng lịch sử cải thiện ROC-AUC và PR-AUC trên internal test.
6. Trong top 100 giao dịch rủi ro, số fraud thật tăng từ 74 lên 83.
7. 17 kiểm tra rò rỉ dữ liệu đều đạt.
8. Pipeline đã có 91 kiểm thử tự động đạt trong workspace phát triển.
9. Dataset, cấu hình, mô hình và kết quả đều có mã kiểm tra để truy vết.

---

8. Sản phẩm đầu ra

Mã nguồn

```
configs/    cấu hình dữ liệu, đặc trưng, chia tập và mô hình
pipelines/  lệnh chạy từng bước
src/        phần xử lý chính
tests/      các kiểm thử tự động
scripts/    lệnh chạy toàn bộ pipeline
```

Dữ liệu và mô hình sinh ra tại máy local

```
data/bronze/
data/silver/
data/offline_store/transaction_features/v1/
data/training/model_dataset/v1/
artifacts/preprocessing/v1/
models/fraud_detection/v1/
```

Báo cáo kết quả

```
reports/generated/model_evaluation/v1/
reports/generated/milestone_seven/v1/
reports/generated/hardening/v1/
```

Project còn có một file ghi lại phiên bản dữ liệu, cấu hình, code, thư viện và mô hình đã tạo ra kết quả. Có thể coi file này là “phiếu lý lịch” của lần chạy.

---

9. Hạn chế

1. Dataset không có mã khách hàng thật nên khóa nhóm chỉ là cách đại diện phục vụ nghiên cứu.
2. Thời gian giao dịch là thời gian tương đối, không phải ngày thật.
3. Project mới tập trung vào lưu đặc trưng để huấn luyện, chưa xây API trả đặc trưng tức thời.
4. Bộ đặc trưng còn nhỏ và chủ yếu dùng để chứng minh quy trình.
5. Phần tối ưu mới giảm dữ liệu trung gian, chưa chứng minh chạy nhanh hơn ở quy mô lớn.
6. Nhóm chỉ có thông tin thẻ chưa cải thiện đồng đều ở mọi chỉ số.
7. Mô hình chưa được tối ưu sâu vì mục tiêu chính là kiểm chứng hệ thống dữ liệu.
8. Kết quả clean-clone dry-run cần hoàn tất trước khi phát hành phiên bản cuối.
9. Kiến trúc Google Cloud hiện là kế hoạch tuần 3–5, chưa được xem là kết quả đã triển khai.

---

10. Kết luận

Đề tài đã xây dựng được một quy trình hoàn chỉnh từ dữ liệu gốc tới đặc trưng lịch sử, dữ liệu huấn luyện, mô hình và báo cáo đánh giá. Điểm quan trọng nhất là mọi đặc trưng lịch sử đều tuân theo thời điểm dự báo, nhờ đó hạn chế việc mô hình nhìn thấy dữ liệu tương lai.

Kết quả mô hình cho thấy đặc trưng lịch sử có thể cung cấp thêm thông tin cho việc phát hiện gian lận. Tuy nhiên, giá trị lớn nhất của project là khả năng kiểm tra, tính lại, theo dõi phiên bản và giải thích nguồn gốc của kết quả.

Nguyên mẫu kỹ thuật đã được xây dựng và kiểm tra trên máy local trong giai đoạn đầu. Trong ba tuần tiếp theo, project sẽ tập trung đóng gói, triển khai lên Google Cloud, tự động hóa lịch chạy và thực hiện demo end-to-end. Sau các bước này, project có thể được đóng gói thành một phiên bản trình bày và bàn giao hoàn chỉnh.

Phụ lục — Tài liệu liên quan

- [`PROJECT_PROGRESS.md`](../PROJECT_PROGRESS.md): theo dõi tiến độ 9 mốc.
- [`IMPLEMENTATION_LOG.md`](./IMPLEMENTATION_LOG.md): giải thích chi tiết các phần đã làm.
- [`pit_time_policy.md`](./pit_time_policy.md): quy tắc sử dụng dữ liệu theo thời gian.
- [`feature_catalog.md`](./feature_catalog.md): danh sách đặc trưng.
- [`model_evaluation.md`](./model_evaluation.md): kết quả đánh giá mô hình.
- [`hardening_report.md`](./hardening_report.md): kiểm tra độ đúng và độ an toàn.
- [`../README.md`](../README.md): hướng dẫn thiết lập và chạy toàn bộ project.